

Task 3:

In OpenCV, we can convert from RGB to grayscale by specifying the enum value from `cv::ColorConversionCodes`. This contains the color conversion codes.

Eg: when using `cv::cvtColor` with `cv::COLOR_BGR2BGRA` : adds transparency value A(alpha) to the BGR channels.

Image 1 shows the color change when using `COLOR_BGR2GRAY` which transforms BGR channels to a monotone.



Image 1: original and cvtColor

Task 4:

To calculate the custom Grayscale filter, I've taken the average across all the BGR channels, and updated the destination frame with the same value.

As we can see in Image 2, the final resultant is very slightly (negligible) darker than the original grayscale image.

```
destRow[c] = static_cast<uint>((px[0]+ px[1]+ px[2]) / 3);
```

Reference:

<https://www.geeksforgeeks.org/python/python-grayscale-of-images-using-opencv/>



Image 2: original, cvtColor and custom averageGrayscaleFilter

To experiment, I've removed the delta (225 - curr value) from the blue channels, and added it with the average. This resulted in a drastic inverse of the grayscale image, where the image is covered with darker background and shadows/darker colors highlighted.

```
uchar diff = 255 - px[0];  
uchar avg = static_cast<uint>((px[0] + px[1] + px[2]) / 3);  
  
destRow[c] = static_cast<uint>(avg + diff);
```



Image 2.1: original, cvtColor and custom averageGrayscaleFilter with blue delta

Since adding the average across the image was making the image darker, I've just retained the delta, and the resultant image was "x-ray" styled.

One of the interesting things I've noticed with this filter was that, when I used it, I was clearly able to see the pimples as it highlighted the bumpy spots. This makes me wonder, is it because the surface is rough, and the green channel was able to pick that difference? Or was it because the affected area is slightly more "red" compared to the surroundings?

```
uchar diff = 255 - px[1];  
destRow[c] = static_cast<uint>(diff);
```



Image 2.2: original, cvtColor and custom applyXRayFilter with only green delta



Task 5:



Image 2.2: original, cvtColor and custom applyXRayFilter with only green delta

For each pixel at row r and column c , the filter computes four values:

Distance from center. The center is $(\text{rows}/2, \text{cols}/2)$, and the pixel's distance from it is $d = \sqrt{dx^2 + dy^2}$.

Normalized distance. d is divided by the center-to-corner distance, $\sqrt{((\text{cols}/2)^2 + (\text{rows}/2)^2)}$, which gives norm in $[0, 1]$: 0 at the center and 1 at the corners. Normalizing makes the effect independent of resolution, so the same settings look the same on a 640×480 webcam frame and a 12-megapixel photo.

Position within the fade band. An inner radius (0.3 by default) defines a central region that isn't changed. Between the inner radius and the corners, the pixel's position is remapped to $t = \text{clamp}((\text{norm} - \text{inner}) / (1 - \text{inner}), 0, 1)$. This is 0 inside the untouched region and 1 at the corners.

Brightness factor. $\text{factor} = 1 - \text{strength} \cdot t^2(3 - 2t)$, where strength is 0.8 by default. The $t^2(3 - 2t)$ term is the smoothstep curve. A linear ramp would change slope abruptly at the inner radius and leave a visible ring. Smoothstep has zero slope at both ends, so the darkening fades in without a visible edge. At the corners, brightness falls to $1 - \text{strength} = 20\%$, so the corners never become fully black.

Each color channel is multiplied by factor and clamped to $[0, 255]$. Because the filter reads the image's channel count while it runs, it works on both color (sepia, RGB) and single-channel (greyscale, X-ray) frames.

After applying vignette, here are the results:

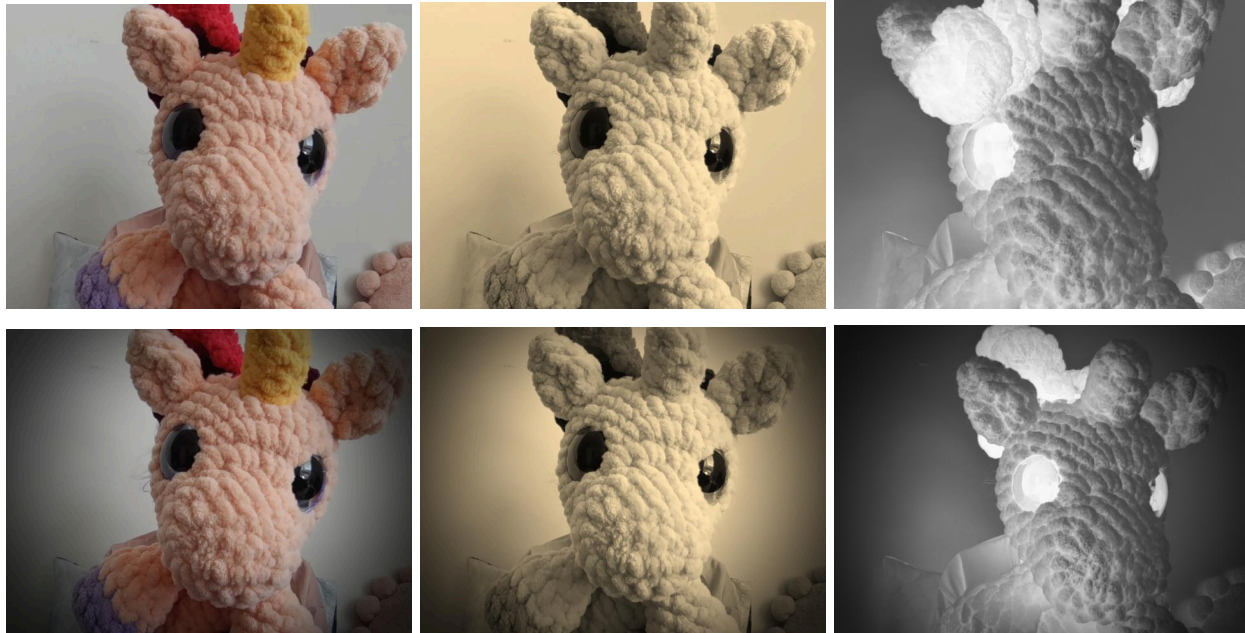


Image 3: original, sepia filter and with vignette on different filters

The vignette works as an independent layer that can go on top of any other effect. Pressing `v` turns it on or off regardless of the current mode, so it can be combined with the original color stream, sepia, the OpenCV greyscale, the custom greyscale or the X-ray filter. These modes produce two kinds of frames: three-channel color images (original and sepia) and single-channel images (both greyscales and X-ray). The filter checks how many channels the incoming frame has while the program runs and applies the same brightness factor to each channel, instead of assuming three.

An earlier version did assume a three-channel color layout for every frame, and it crashed as soon as the vignette was turned on over a single-channel greyscale frame, because it read past the end of the smaller image buffer. Making the filter channel-agnostic fixed the crash and let the vignette work with every mode. Because the darkening factor depends only on a pixel's position and not on its color, the effect looks the same whether it's applied to a color or a greyscale frame.

Task 6:

Both versions blur each color channel with the integer Gaussian kernel, a 5×5 grid of weights that sum to 100. Each output pixel is the weighted average of the 25 pixels around it. The input is copied to the output first, so the outer two rows and columns keep their original values, and sums are kept in integers because they can reach 25,500 before dividing by 100.

6A (naive) computes all 25 products for every pixel and reads each pixel with OpenCV's `at` accessor, which recalculates the row address and checks bounds on every call.

6B (separable) uses the fact that the kernel is the product of the 1D filter `[1 2 4 2 1]` with itself. A horizontal pass blurs the rows into a 16-bit temporary image (sums up to 2,550 don't fit in 8 bits), and a vertical pass blurs its columns, which costs 10 operations per pixel instead of 25. It divides by 100 once at the end, so the result matches 6A exactly [verified: max pixel difference 0]. It also replaces `at` with row pointers that are fetched once per row.

640×480 image, Debug build Time per image

6A: naive, at **0.5193 s**

6B: separable, row pointers **0.0810 s** (6.4× faster)

The speed-up comes from two roughly equal factors: 2.5× less arithmetic from separability, and about 2.5× from removing `at`'s per-pixel overhead.

```
→ video-special-effects git:(main) 🌸 🍄 🎈 ./out/timeBlur.exe data/cathedral.jpeg
Time per image (1): 0.5193 seconds
Time per image (2): 0.0810 seconds
Terminating
→ video-special-effects git:(main) 🌸 🍄 🎈
```

6B runs live on the video stream with the `b` key.

Since the kernel size is 5×5, the blur is not hugely distinguishable on larger images.



Image 4: original, 5x5 gaussian blurred image

Task 7,8:

Sobel X / Y (Task 7). Each 3×3 Sobel is applied as two separable 1×3 passes: $X = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}^T \times \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$, $Y = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}^T \times \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$. This takes 6 multiplies per pixel per channel instead of 9. Pixels are accessed through row pointers, and no OpenCV filter functions are used.

X computes right – left, so it is positive to the right. Y computes above – below, so it is positive upward (row indices grow downward).

The result is divided by 4, the sum of the smoothing weights, to keep it in $[-255, 255]$, and stored as CV_16SC3.

For display, convertScaleAbs shows $|\text{value}|$, so edges of either sign appear bright. The signed output is kept in a separate variable from the displayed image.

As expected, X highlights vertical edges and Y highlights horizontal edges.

Gradient magnitude (Task 8). $\text{magnitude}()$ computes $\sqrt{s_x^2 + s_y^2}$ separately for each channel, giving a CV_8UC3 color image that shows edges in every orientation. The maximum possible value is about 360 ($255 \cdot \sqrt{2}$). Values above 255 are clamped rather than rescaled, because rescaling would dim typical edges by about 30%.

Keys: 1 Sobel X, 2 Sobel Y, 3 magnitude.

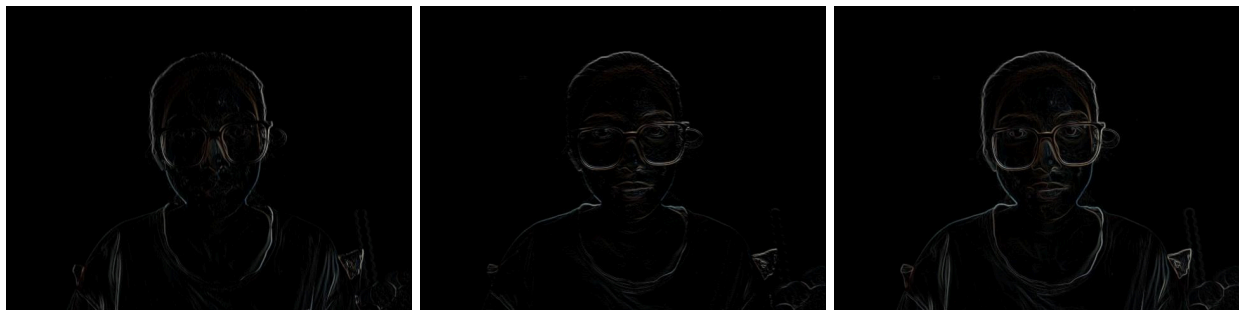


Image 5: X-Sobel, Y-Sobel, Magnitude (X-sobel+Y-sobel) filter

Task 9:

$\text{blurQuantize}(\text{src}, \text{dst}, \text{levels})$ reuses the task 6 separable blur ($\text{blur}_{5 \times 5_2}$), then quantizes each channel in place. The bucket size is $b = 255 / \text{levels}$, and each value becomes $x / b * b$, which rounds it down to the bottom of its bucket.

Blur first: without it, sensor noise pushes pixels back and forth across bucket boundaries, which shows up as speckles in still images and flicker in video.

Result: flat, posterized color regions with a cartoon-like look. With levels = 10, $b = 25$, so values 125–149 all become 125.

Level count: because of integer division, values 250–255 form an 11th bucket, so each channel ends up with at most 11 values rather than exactly 10.

Input check: levels must be between 1 and 255. At 0 the bucket size calculation divides by zero, and above 255 the bucket size becomes 0.

Key: l (10 levels).

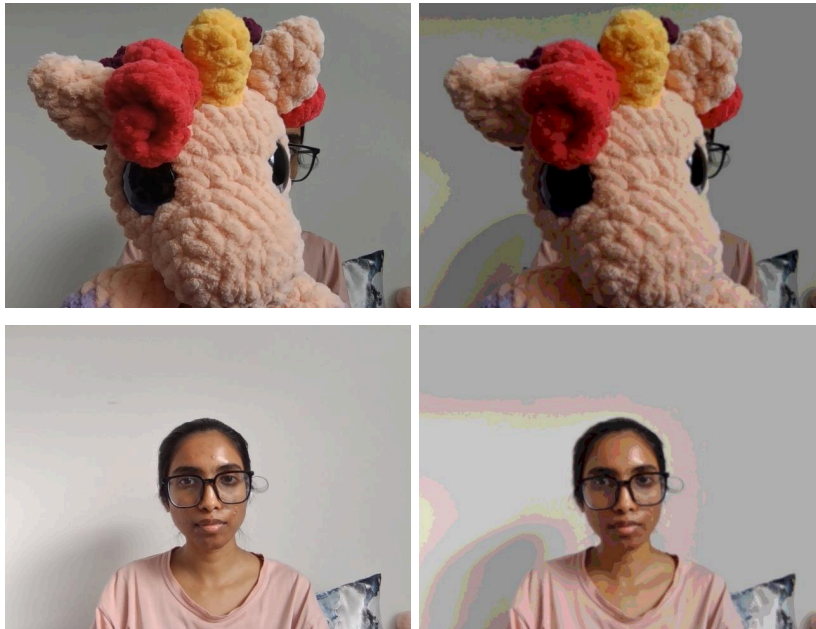


Image 6: Original and quantized image

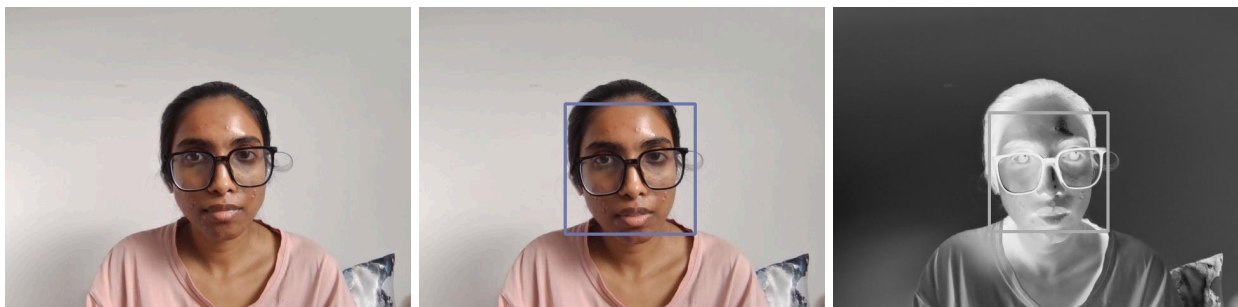
Task 10:

Detect on the original frame: faces are always found in the unfiltered camera frame, converted to greyscale. The Haar detector expects a natural image, so running it on filter output such as Sobel edges would fail.

Draw on what's displayed: the boxes are drawn on whatever the active filter produces, so face detection works on top of every mode.

Reused later: the face boxes also drive the task 12 rain effect, which keeps the face clear.

Key: f (toggle).



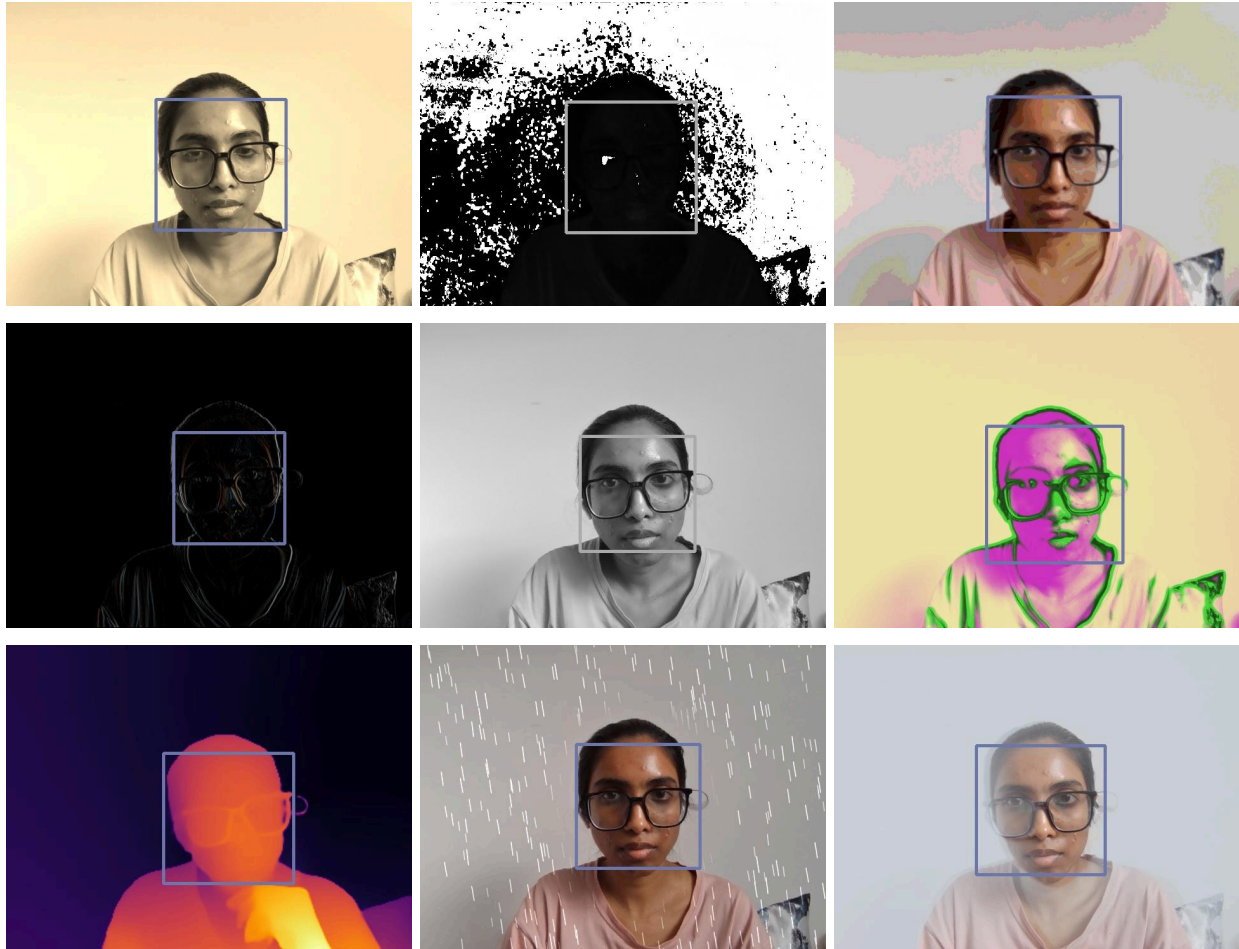


Image 7: Face detection with filters

Task 11:

Depth Anything V2 (model_fp16.onnx) runs through ONNX Runtime on the CPU, using the provided DA2Network wrapper. Its output is a CV_8UC1 map where 255 means nearest.

Input size: each frame is scaled to $0.5 \times (320 \times 240)$ before going into the network, as a balance between speed and depth accuracy.

Refresh rate: DA2 is slow on the CPU, so depth is recomputed every 3rd frame and reused in between. People move little in that time.

Lazy loading: the network is loaded the first time a depth mode is selected, so the other modes don't pay the startup cost.

Display: the depth map is shown with the INFERNO colormap, where bright yellow means near.

Creative filter: depth fog. Each pixel is blended toward a light grey fog color by $f = 1 - \exp(-\text{density} \cdot \text{distance})$, where $\text{distance} = 1 - \text{depth}/255$. This follows the Beer-Lambert

law real fog obeys: nearby objects stay clear and far objects fade into haze. The default density is 2.5.

Stabilization: DA2's min-max normalization shifts from frame to frame as things move, which made the fog pulse. A running average of the depth map (70% previous, 30% new) removes that.

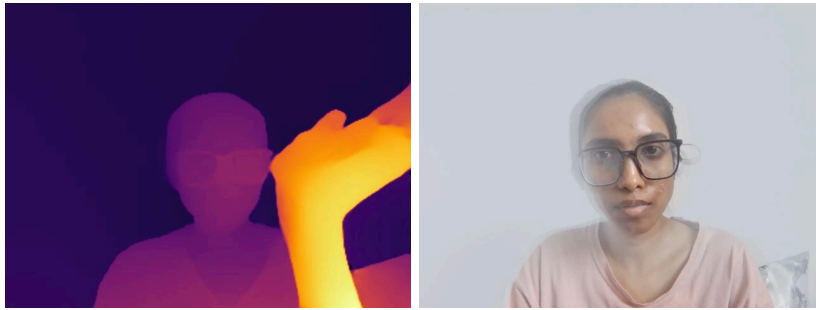


Image 8: Image showing the depth of the frame and fog filter



Image 9: Rain filter using the depth

Task 12:

1. Seattle Rain (uses face detection), key r. Animated, see-through rain streaks are added on top of the frame's pixel values, so the scene stays visible through them. Around each detected face the rain fades out through a soft oval with a gradual edge, so it never rains on the user's face. The oval is averaged over frames so it doesn't wobble with the detector. Each drop has a pseudo-depth: nearer drops fall faster and are longer and brighter, which gives a parallax effect.

2. Neon duotone (area-based), key n. A detail map is built from my Sobel X/Y and gradient magnitude functions, then lightly blurred so textured regions (face, hands, clothing) score high and smooth walls score low. Textured regions are colored in a dark-to-light green duotone by brightness, with strong edges pushed to bright green for an outline rim. Smooth regions get a magenta to pale-yellow duotone, and the two palettes blend softly where they meet.

3. Cartoonize (area-based), key t. The flat colors come from my blurQuantize (task 9), and black ink outlines are drawn wherever the gradient magnitude (task 8) exceeds a threshold. Two details make it work on live video:

The edges are computed on a blurred copy of the frame, so webcam noise doesn't turn into black speckles.

A pixel's edge strength is the strongest of its three channels, so boundaries between colors of similar brightness still get a line.

How the requirements are met. Rain builds on face detection; neon and cartoonize use area computations. All three reuse my own earlier functions (blur, Sobel, magnitude, quantize). OpenCV appears only as one step inside a larger process: GaussianBlur and cvtColor in neon, line for drawing rain streaks.



Image 10: Median filter



Image 11: Cartoon filter

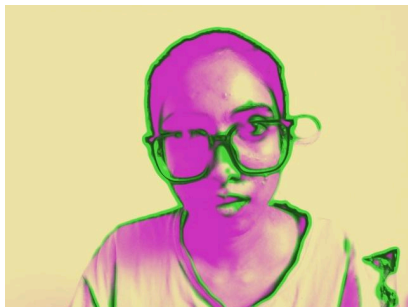


Image 12: Neon Magnitude filter



▶ Strobe Disco

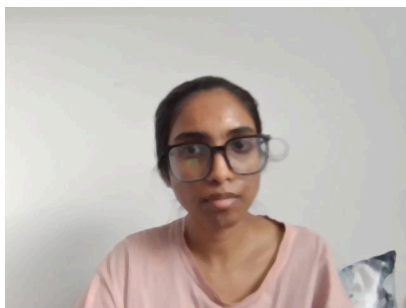
Image 13: Disco Strobe filter

Extensions:

Vignette (key v, toggle). The frame darkens toward the corners, as with an old camera lens. Each pixel's distance from the center is measured as a fraction of the center-to-corner distance, so the effect looks the same at any resolution. Pixels within 30% of that distance are untouched, and beyond that brightness falls to 20% at the corners. The falloff uses smoothstep, which avoids the visible ring a linear fade leaves where it starts. The vignette works on 1- and 3-channel images and is applied last, so it can be layered on any mode, including greyscale and X-ray.



Median filter (key m). Each channel value is replaced by the median of its 5×5 neighborhood, implemented from scratch. A Gaussian blur averages across edges; the median removes speckle noise but keeps edges sharp, and at larger window sizes it gives a flat, painted look. Each median is found with `std::nth_element`, which only has to place the middle value instead of sorting all 25, so each pixel costs $O(n)$ rather than $O(n \log n)$. The outer 2 rows and columns are copied from the original frame.



Disco lights (key k, uses face detection). Each pixel keeps its brightness but is recolored with saturated neon hues in diagonal rainbow bands that sweep across the frame, cycling every 2 seconds. A strobe alternates between full and dim brightness at 2.5 Hz, kept below the 3 Hz photosensitive-seizure guideline. The detected face keeps its natural color through the same soft oval mask the rain effect uses, while the background strobes; pressing k again swaps the two. The animation is driven by the clock rather than a frame count, so it runs at the same speed at any frame rate.



LLM Usage

I used Claude (Anthropic, via Claude Code in VS Code) as a coding and writing assistant.

Code written with Claude: Claude implemented the cartoonize filter, the median filter extension and the disco extension, including moving the soft face-oval mask into a helper shared by the rain and disco effects, and it added their keyboard modes to vidDisplay.cpp. I specified what each effect should do and chose which three effects count toward Task 12.

Report: Claude drafted the descriptions for Tasks 7–12 from my code and the assignment text. I reviewed them and chose the images.

Debugging and set up: Claude helped me debug the rain/neon/fog filters, set up CMake/ONNX and connect Face detection by downgrading the project from OpenCV 5 to OpenCV4 setup

I reviewed all generated code, built it, and tested it on my webcam before submitting.